

Assignment 4: Sri Tel - New Customer Experience

Service Oriented Computing (CSS4242/CSC4222)

Group Members:

H.S.R. Yasas (SC/2020/11431)

T.G.M.M.Ferdinandaz (SC/2020/11446)

February 7, 2026

Contents

1	Introduction	2
2	Proposed Architectures	2
2.1	Alternative Architecture 1: Service-Oriented Architecture (SOA) with ESB . . .	2
2.2	Alternative Architecture 2: Microservices Architecture (Selected)	3
2.2.1	Conceptual Architecture	3
2.2.2	Implementation Architecture	3
2.3	Rationale for Selection	4
3	Architectural and Integration Patterns	4
3.1	Patterns Used	4
3.2	Rationale for Pattern Selection	5
4	Information Security Considerations	5
5	Conclusion	5

1 Introduction

Sri Tel Ltd (STL), a telecommunication provider offering GSM/3G/4G/LTE services, aims to increase market share by improving customer care and experience. The current initiative involves deploying a state-of-the-art "Sri-Care" solution comprising a web portal and smartphone applications (iOS/Android).

The primary objective of this solution is to allow existing customers to manage their accounts, pay bills, and activate services without manual intervention from support staff. The solution must integrate with an existing Provisioning System and an external Payment Gateway while ensuring scalability, security, and high availability for notification delivery.

2 Proposed Architectures

To address the requirements of scalability, multi-client support (Web & Mobile), and integration with external systems, we analyzed two distinct architectural approaches.

2.1 Alternative Architecture 1: Service-Oriented Architecture (SOA) with ESB

The first approach utilizes a traditional Service-Oriented Architecture centered around an Enterprise Service Bus (ESB).

Description: In this model, an ESB acts as the central backbone. All client requests are routed through the ESB, which orchestrates calls to various backend services. The ESB handles message transformation, protocol bridging, and routing.

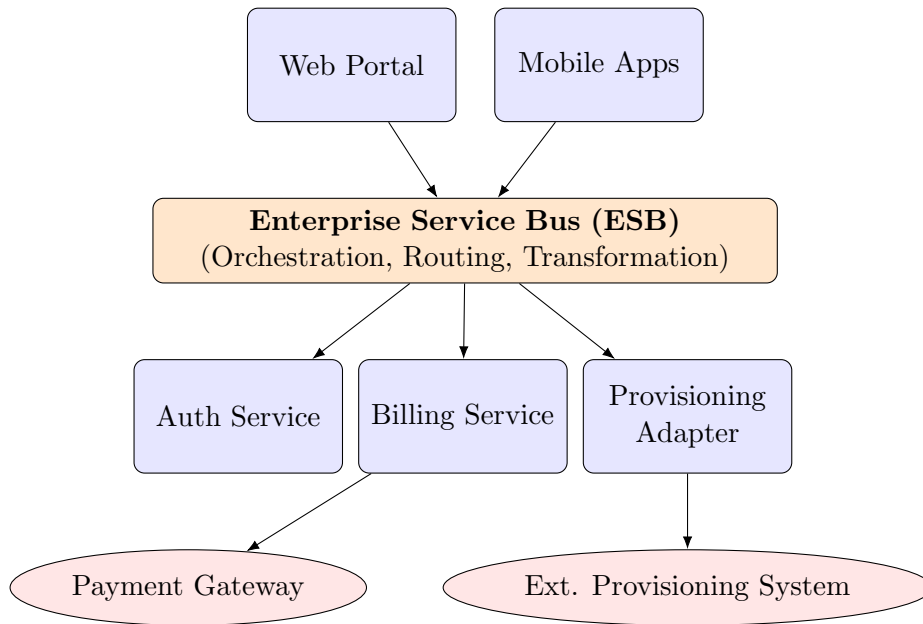


Figure 1: Alternative 1: SOA with ESB

Pros and Cons:

- **Pros:** Strong integration capabilities; centralized management of policies; mature technology for protocol mediation.
- **Cons:** The ESB can become a single point of failure and a performance bottleneck; logic often bleeds into the ESB (smart pipes), making it harder to scale individual components; heavy-weight for simple mobile app backends.

2.2 Alternative Architecture 2: Microservices Architecture (Selected)

The second and selected approach is a Microservices Architecture utilizing an API Gateway and Event-Driven asynchronous communication for notifications.

2.2.1 Conceptual Architecture

The application is broken down into small, independent services (User, Bill, Notification, Provisioning). An API Gateway sits in front of the services, handling routing, authentication, and rate limiting. Inter-service communication is primarily RESTful for synchronous operations and Message-based (Pub/Sub) for asynchronous tasks like notifications.

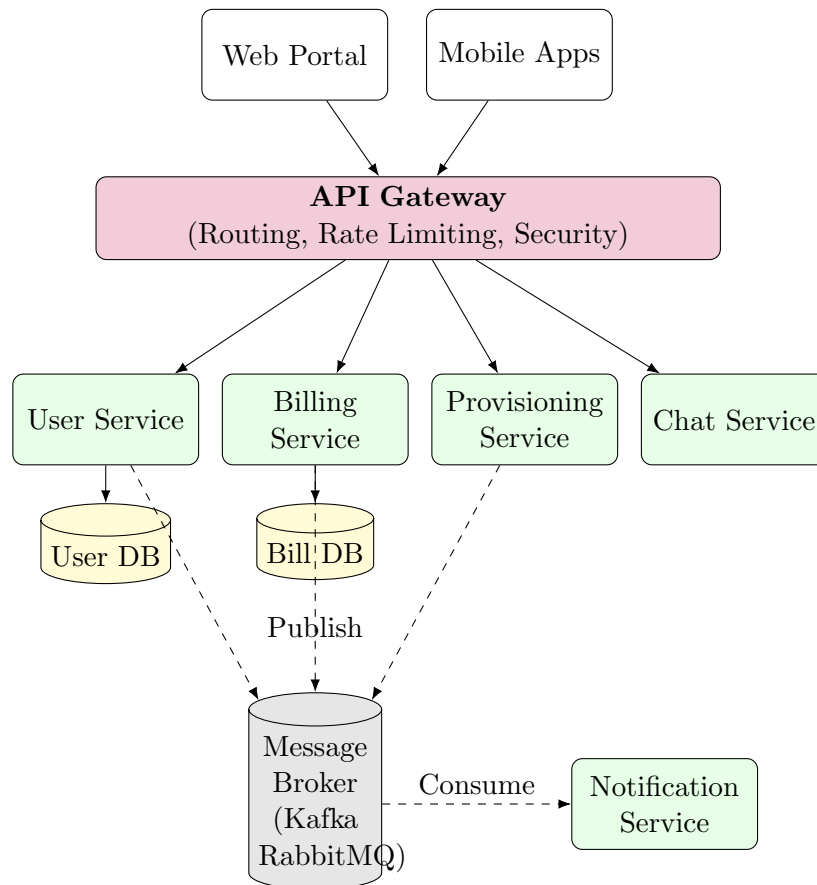


Figure 2: Conceptual View: Microservices with Event-Driven Notifications

2.2.2 Implementation Architecture

The realized implementation translates the conceptual design into a containerized deployment model using industry-standard open-source technologies.

- **Containerization:** Docker and Docker Compose are used to package services and their dependencies (PostgreSQL), ensuring consistent environments.
- **Backend Services:** Implemented using **Java 17** and **Spring Boot** for rapid development and robustness.
- **Database:** **PostgreSQL** is used as the relational database, running in a dedicated container.

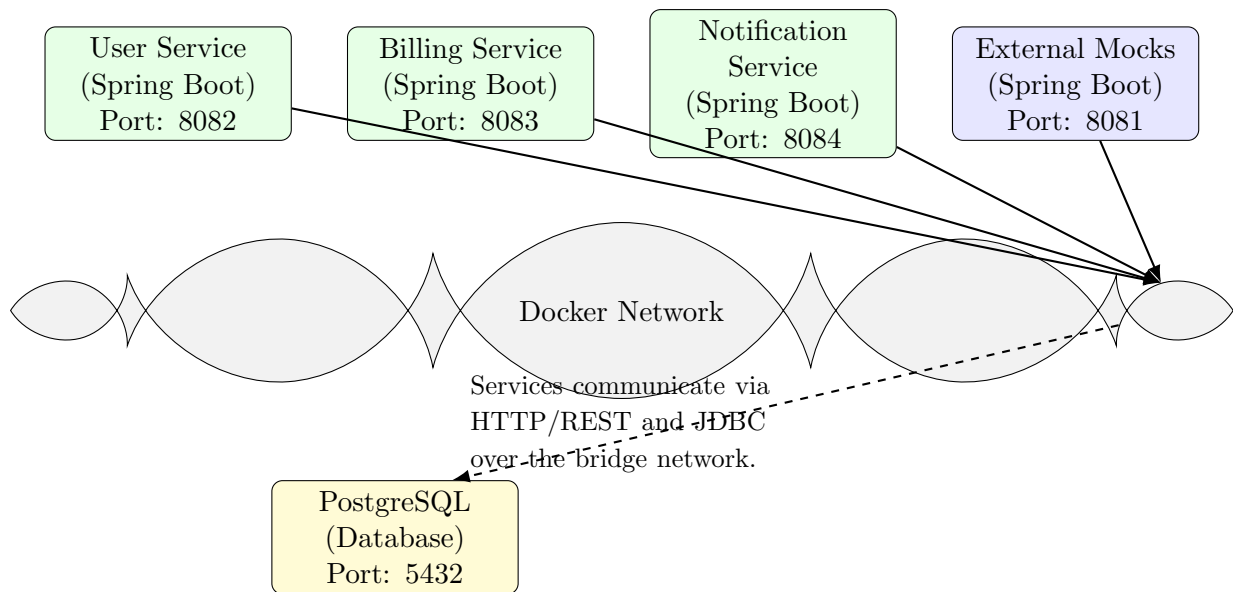


Figure 3: Implementation View: Docker Container Deployment

2.3 Rationale for Selection

We selected the **Microservices Architecture** for the Sri-Care solution for the following reasons:

- **Scalability:** The notification service is expected to handle high volumes ("best effort"). Microservices allow us to scale the Notification Service and Message Broker independently of the User or Billing services.
- **Agility:** The "Sri-Care" portal and apps require frequent updates. Independent deployment of microservices allows faster iteration without redeploying the entire monolith.
- **Fault Isolation:** A failure in the Chat service will not impact the critical Billing or Provisioning functionalities.

3 Architectural and Integration Patterns

Referencing the course notes on Microservices and Integration Patterns, the following patterns are implemented:

3.1 Patterns Used

- **Architectural Patterns:**
 - **API Gateway:** Single entry point for all clients (Web and Mobile).
 - **Database per Service:** Each microservice owns its database schema to ensure loose coupling.
 - **Backend for Frontend (BFF):** Optimized APIs for specific client types.
- **Integration Patterns:**
 - **Publish-Subscribe Channel:** For asynchronous notifications.
 - **Adapter Pattern:** For integrating with the legacy Provisioning System.

3.2 Rationale for Pattern Selection

- **API Gateway:** Chosen to decouple the client apps from the internal microservices structure. It simplifies the client code (one endpoint) and centralizes cross-cutting concerns like Authentication (OAuth).
- **Publish-Subscribe:** Selected specifically for the "Best Effort" notification requirement. It decouples the Critical Path (Billing) from the Non-Critical Path (Notification), ensuring that a slow email server does not block a user's payment confirmation.
- **Database per Service:** Selected to prevent "Integration Database" anti-patterns. It allows the Billing team to modify their schema without breaking the User service, facilitating independent development.

4 Information Security Considerations

During the design and implementation, the following security aspects were prioritized:

- **Authentication & Authorization (OAuth 2.0 / OIDC):** Using an Identity Provider (IdP) like Keycloak. The Mobile App and Web Portal authenticate against the IdP and receive an Access Token (JWT). The API Gateway validates this token before forwarding requests.
- **Transport Security (TLS/HTTPS):** All communication between clients and the Gateway, and between internal services, is encrypted.
- **Network Isolation:** The Microservices and Databases reside in a private Docker subnet, not directly accessible from the internet. Only the API Gateway is exposed publicly.
- **Data Privacy:** User passwords are not stored in plain text but are hashed (mocked in prototype, required for production).

5 Conclusion

The proposed Microservices solution, implemented using Spring Boot and Docker, effectively addresses Sri Tel's needs for a modern, scalable customer care platform. By leveraging the API Gateway for unified access and an Event-Driven architecture for high-volume notifications, the system ensures robustness and a superior user experience.